

Android Application 生命周期及应用示例

Android Application 生命周期（简单明了版）

核心结论：**Application 是 App 全局单例，生命周期与 App 进程绑定，仅初始化一次，销毁随进程终止，主要用于全局初始化、资源管理。**

一、核心生命周期方法（仅3个关键方法）

Application 生命周期极简，仅需关注以下3个回调（按执行顺序），无需布局XML，仅需在代码中重写：

方法名	调用时机	核心作用	注意事项
<code>onCreate()</code>	App 进程创建时调用（全局仅1次）	初始化全局资源（如网络、数据库、第三方SDK）	不能做耗时操作（启动变慢）
<code>onConfigurationChanged()</code>	设备配置发生变化时（如屏幕旋转、语言切换）	适配配置变更（如重新加载布局资源）	需在 AndroidManifest 中声明要监听的配置重启App
<code>onTerminate()</code>	App 进程终止时调用（仅模拟器有效）	理论上释放资源（如关闭连接）	真实设备上几乎不杀进程时不会回调

二、XML 配置（仅需注册Application子类）

若需自定义 Application（重写生命周期方法），需在 `AndroidManifest.xml` 中注册，无需额外复杂配置：

Code block

```
1  <!-- 1. 声明自定义 Application 子类（核心配置） -->
2  <application
3      android:name=".MyApplication"  <!-- 替换为你的 Application 子类全路径 -->
4      android:icon="@mipmap/ic_launcher"
5      android:label="@string/app_name"
6      android:theme="@style/Theme.MyApp">
7
8      <!-- 2. 可选：配置监听的设备配置（触发 onConfigurationChanged） -->
9      <activity
10         android:name=".MainActivity"
```

```

11         android:configChanges="orientation|screenSize|locale"> <!-- 监听屏幕旋
    转、尺寸、语言变化 -->
12         <intent-filter>
13             <action android:name="android.intent.action.MAIN" />
14             <category android:name="android.intent.category.LAUNCHER" />
15         </intent-filter>
16     </activity>
17 </application>

```

三、自定义 Application 示例代码（含生命周期回调）

Code block

```

1  public class MyApplication extends Application {
2      // 全局单例，可通过 getInstance() 在任意地方获取
3      private static MyApplication instance;
4
5      @Override
6      public void onCreate() {
7          super.onCreate();
8          instance = this;
9          // 1. 初始化全局资源（仅执行1次）
10         initRetrofit(); // 初始化网络请求SDK
11         initRoom();      // 初始化数据库
12         initGlide();      // 初始化图片加载库
13         Log.d("AppLife", "Application onCreate() - App进程创建");
14     }
15
16     @Override
17     public void onConfigurationChanged(Configuration newConfig) {
18         super.onConfigurationChanged(newConfig);
19         // 2. 适配配置变更（如屏幕旋转后重新设置资源）
20         if (newConfig.orientation == Configuration.ORIENTATION_LANDSCAPE) {
21             Log.d("AppLife", "屏幕横屏");
22         } else if (newConfig.orientation ==
23 Configuration.ORIENTATION_PORTRAIT) {
24             Log.d("AppLife", "屏幕竖屏");
25         }
26         Log.d("AppLife", "Application onConfigurationChanged() - 配置变更");
27     }
28
29     @Override
30     public void onTerminate() {
31         super.onTerminate();
32         // 3. 仅模拟器有效，真实设备几乎不触发
33         Log.d("AppLife", "Application onTerminate() - 进程终止（仅模拟器）");

```

```
33     }
34
35     // 全局获取 Application 实例（方便跨组件调用）
36     public static MyApplication getInstance() {
37         return instance;
38     }
39
40     // 示例：初始化第三方库（实际根据需求添加）
41     private void initRetrofit() {}
42     private void initRoom() {}
43     private void initGlide() {}
44 }
```

四、关键使用场景

1. **全局初始化**：网络SDK（Retrofit）、数据库（Room）、图片加载（Glide）、推送SDK等，避免在Activity中重复初始化；
2. **全局状态管理**：存储App全局变量（如用户Token、设备信息），通过 `getInstance()` 在任意Activity/Fragment中访问；
3. **配置变更适配**：屏幕旋转、语言切换时，无需重启Activity，直接在 `onConfigurationChanged()` 中处理资源适配；
4. **内存监控**：可结合 `onLowMemory()`（低内存时回调）做资源释放（如清理图片缓存），优化App性能。

五、避坑要点

1. **禁止耗时操作**：`onCreate()` 是App启动的关键流程，耗时操作（如下载、数据库迁移）会导致启动白屏；
2. **不要存储大量数据**：Application 是单例，长期持有大对象（如Bitmap、大集合）会导致内存泄漏；
3. **`onTerminate()` 别依赖**：真实设备中，系统杀进程时不会调用该方法，资源释放需在Activity/Fragment的生命周期中处理；
4. **必须注册XML**：自定义 Application 后，若未在 AndroidManifest.xml 中配置 `android:name`，会使用系统默认 Application，自定义方法不生效。

（注：文档部分内容可能由 AI 生成）